

ResNet-18 Inference on RISC-V RVV 1.0

IREE compilation and QEMU execution on the CAID ChipCluster

Host: chip01

User: school26

Working directory: /shared/school26/resnet18_iree

Validation date: 4 September 2026

FINAL STATUS VALIDATED — the compiled VMFB executed successfully under QEMU and produced a finite 1×1000 float32 output tensor.

Executive Summary

The objective was to establish a reproducible end-to-end path from ResNet-18 MLIR to an IREE VM flatbuffer (VMFB) targeting 64-bit RISC-V with vector support, then execute it through QEMU using the correct RISC-V IREE runtime. The work also had to be performed in a directory writable by the school26 account without weakening shared project permissions.

The final configuration succeeded. QEMU returned status 0, the runtime reported EXEC @main_graph, and NumPy validation confirmed shape (1, 1000), data type float32, and only finite values. Because the input was an all-zero tensor, this is a functional toolchain validation, not an ImageNet accuracy result.

VMFB	EXECUTION	OUTPUT	VALIDATION
45 MB RISC-V/RVV target module	Status 0 EXEC @main_graph	1×1000 float32; 4.1 KB	PASSED All values finite

SCOPE BOUNDARY This report proves model compilation, module loading, RISC-V emulation, runtime execution, and output integrity. A real-image test and timing study are separate next steps.

1. Problem Statement and Diagnostic Findings

The initial workflow encountered several independent issues. Resolving them in sequence was essential because a successful compilation does not guarantee that the selected runtime can load or execute the VMFB.

Stage	Observed evidence	Root cause	Resolution
Workspace	Permission denied and rsync code 23 for old logs/VMFB files	The shared source tree contained protected artifacts	Continue in the user-owned /shared/school26 copy
First run	No .npy output was created	\$HOME/iree-build-riscv/tools/iree-run-module did not exist	Locate actual binaries and verify their architecture
Runtime test	required module features [Ch] are not available	The shared RISC-V runtime and compiled VMFB were incompatible	Use the compatible /opt/iree-run-module runtime
QEMU option	Uppercase Zve64d deprecation warning	Property spelling changed in QEMU	Use lowercase zve64d
Final run	Status 0; EXEC @main_graph; output created	Compiler, guest runtime, QEMU, and sysroot were aligned	Retain the working paths as the supported configuration

2. Confirmed Working Configuration

Component	Confirmed path or value	Role
Project	/shared/school26/resnet18_iree	Writable user workspace
Compiler	/usr/local/bin/iree-compile	Converts MLIR to VMFB
Runtime	/opt/iree-run-module	RISC-V guest executable compatible with the VMFB
QEMU	/opt/riscv/bin/qemu-riscv64	Emulates the RISC-V CPU
Sysroot	/opt/iree-riscv-toolchain/sysroot	Provides the RISC-V loader and shared libraries
Target	RV64; lp64d; +m,+a,+f,+d,+c,+v,+zvl256b	RISC-V scalar and vector feature set
Vector model	RVV 1.0; VLEN 256; ELEN 64	QEMU vector execution configuration

DO NOT USE /shared/compilers/iree/build-riscv/install/bin/iree-run-module for this VMFB. It launched under QEMU but rejected the module because of an IREE bytecode feature mismatch.

3. End-to-End Working Flow

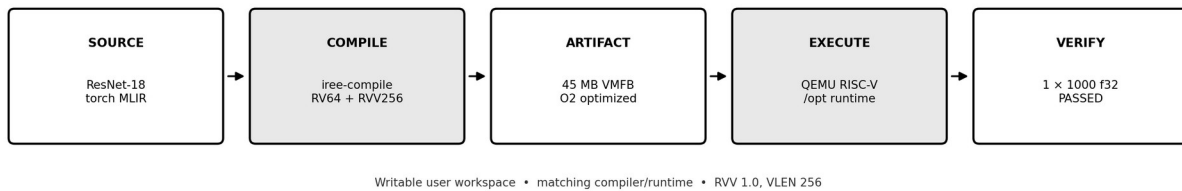


Figure 1. Verified ResNet-18 compilation and execution path

4. Step-by-Step Procedure

Step 1. Connect and enter the writable project

What we did: We used the school26 account and worked under its own directory in /shared.

Why: The original /shared/projects tree was a shared reference location and contained files the account could not overwrite.

Importance: A user-owned workspace prevents permission failures, preserves the original project, and avoids unsafe chmod 777 changes.

```
ssh school26@chip01
cd /shared/school26/resnet18_iree
pwd
ls
```

OBSERVED RESULT The active directory was /shared/school26/resnet18_iree and contained export_resnet18.py, input, logs, mlir, model, output, test_images, and vmfb.

SAFE STAGING FOR A NEW USER If a clean copy is required, copy source material while excluding protected generated artifacts. Do not change the permissions of the shared master project.

```
mkdir -p /shared/school26/resnet18_iree
rsync -a \
  --exclude='logs/' \
  --exclude='vmfb/' \
  --exclude='output/' \
  /shared/projects/resnet18_iree/ \
  /shared/school26/resnet18_iree/
```

Step 2. Verify write access before compiling

What we did: We created the required output directories, inspected ownership and permissions, and performed a reversible write test.

Why: This isolates filesystem problems before they are confused with compiler or runtime problems.

Importance: Compilation and inference both create files; confirmed write access makes the run deterministic and protects shared data.

```
cd /shared/school26/resnet18_iree
mkdir -p input output logs vmfb
ls -ld . input output logs vmfb
touch output/write_test
rm output/write_test
```

OBSERVED RESULT The project and output directories were accessible by school26; the write test completed successfully.

Step 3. Compile ResNet-18 MLIR for RISC-V RVV256

What we did: We compiled mlir/resnet18_torch.mlir with the LLVM CPU backend, RISC-V 64-bit triple, lp64d ABI, vector extensions, a 256-bit minimum vector length, LMUL up to 8, and O2 optimization.

Why: These options describe the intended guest architecture and cause IREE to produce a deployable VMFB instead of a host x86 module.

Importance: The VMFB is the portable IREE deployment artifact that connects the model compiler to the target runtime.

```
cd /shared/school26/resnet18_iree
set -o pipefail

iree-compile mlir/resnet18_torch.mlir \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-triple=riscv64 \
  --iree-llvmcpu-target-cpu=generic-rv64 \
  --iree-llvmcpu-target-abi=lp64d \
  --iree-llvmcpu-target-cpu-features="+m,+a,+f,+d,+c,+v,+zvl256b" \
  --riscv-v-fixed-length-vector-lmul-max=8 \
  --iree-opt-level=O2 \
  -o vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb \
  2>&1 | tee logs/resnet18_rvv256_compile.log

COMPILE_STATUS=${PIPESTATUS[0]}
echo "IREE compile status: $COMPILE_STATUS"
test "$COMPILE_STATUS" -eq 0 || exit "$COMPILE_STATUS"
```

OBSERVED RESULT Compilation produced vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb with a size of approximately 45 MB.

Step 4. Confirm the compiled artifact

What we did: We verified that the VMFB existed and was non-empty rather than treating a quiet terminal or an empty log as proof of success.

Why: IREE compilation can complete without verbose output; the artifact and captured command status are the reliable evidence.

Importance: This check prevents a later QEMU failure from being incorrectly diagnosed as a runtime problem when the module is absent.

```
ls -lh vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb
test -s vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb \
&& echo "VMFB creation: PASSED"
```

OBSERVED RESULT A non-empty 45 MB VMFB was present; compilation was therefore accepted as successful.

5. Runtime Diagnosis and Final Execution

Step 5. Locate and identify available IREE runners

What we did: After the original \$HOME/iree-build-riscv path failed, we enumerated all iree-run-module files and used file -L to distinguish Python wrappers, x86-64 executables, and RISC-V ELF binaries.

Why: A filename alone does not prove that a binary has the correct architecture or bytecode compatibility.

Importance: Architecture inspection prevents attempts to execute host binaries as RISC-V guests and makes runtime selection auditable.

```
type -a iree-compile iree-run-module qemu-riscv64 2>/dev/null

find /shared /opt /usr/local "$HOME" \
-type f -name 'iree-run-module' -print 2>/dev/null |
while IFS= read -r candidate; do
  file -L "$candidate"
done
```

OBSERVED RESULT The search found RISC-V ELF runners under /shared and /opt, a Python wrapper at /usr/local/bin, and x86-64 runtime binaries inside the Python package.

Why the first RISC-V runner still failed

The runner at /shared/compilers/iree/build-riscv/install/bin/iree-run-module was a valid RISC-V ELF and started under QEMU. However, it rejected the module with the message

below. This proved that the remaining issue was IREE compiler/runtime compatibility, not QEMU or filesystem access.

```
INVALID_ARGUMENT; required module features [Ch] are not available
in this runtime configuration; have [EXT_F32|EXT_F64] while the
module requires [Ch]
```

TECHNICAL LESSON A target executable can have the correct RISC-V architecture and still be incompatible with a VMFB produced by another IREE revision or runtime feature configuration. Compiler and runtime compatibility must be tested together.

Step 6. Select the compatible runner and define exact paths

What we did: We selected /opt/iree-run-module, the /opt RISC-V QEMU binary, and the /opt RISC-V sysroot, then assigned absolute paths to all artifacts.

Why: Absolute paths eliminate ambiguity from PATH order and prevent the Python wrapper at /usr/local/bin from being selected accidentally.

Importance: This creates a stable, repeatable execution configuration for other users and future sessions.

```
cd /shared/school26/resnet18_iree

RUNNER="/opt/iree-run-module"
QEMU="/opt/riscv/bin/qemu-riscv64"
SYSROOT="/opt/iree-riscv-toolchain/sysroot"
MODULE="$PWD/vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb"
RESULT="$PWD/output/resnet18_rvv256_opt_output.npy"
RUN_LOG="$PWD/logs/resnet18_rvv256_opt_runner.log"

file "$RUNNER"
test -x "$RUNNER" || exit 1
test -s "$MODULE" || exit 1
test -e "$SYSROOT/lib/ld-linux-riscv64-lp64d.so.1" || exit 1
```

OBSERVED RESULT The runner was identified as a dynamically linked 64-bit RISC-V ELF using the lp64d loader available in the selected sysroot.

Step 7. Execute the VMFB under QEMU

What we did: We launched the RISC-V runner with RVV 1.0, a vector length of 256 bits, and an element length of 64 bits. IREE used the local-sync device, called main_graph, and received a zero-filled $1 \times 3 \times 224 \times 224$ float32 tensor.

Why: The zero tensor removes image-preprocessing variability and provides a controlled functional smoke test.

Importance: A successful run validates module loading, guest dynamic linking, runtime initialization, function dispatch, tensor execution, and output writing.

```
"$QEMU" \  
-cpu rv64,zve64d=true,vlen=256,elen=64,vext_spec=v1.0 \  
-L "$SYSROOT" \  
"$RUNNER" \  
--device=local-sync \  
--module="$MODULE" \  
--function=main_graph \  
--input="1x3x224x224xf32=0" \  
--output="@RESULT" \  
>"$RUN_LOG" 2>&1  
  
RUN_STATUS=$?  
echo "QEMU/IREE status: $RUN_STATUS"  
sed -n '1,120p' "$RUN_LOG"  
test -s "$RESULT" && ls -lh "$RESULT"
```

OBSERVED RESULT QEMU/IREE status was 0, the log contained EXEC @main_graph, and a 4.1 KB NumPy result file was created.

Step 8. Validate the output tensor

What we did: We loaded the .npy file with NumPy, inspected its shape and data type, checked that every value was finite, reported the top five indices, and used assertions as an explicit pass/fail gate.

Why: A created file can still be malformed or contain NaN/Inf values; structural and numerical checks are required.

Importance: The assertions convert an informal observation into a repeatable validation test suitable for training, regression testing, and future automation.

```
python3 - <<'PY'
import numpy as np

result = np.load('output/resnet18_rvv256_opt_output.npy')
values = result.reshape(-1)
top5 = np.argsort(values)[-5:][::-1]

print('Shape:', result.shape)
print('Data type:', result.dtype)
print('All finite:', bool(np.isfinite(result).all()))
print('Predicted class index:', int(top5[0]))
print('Top-5 class indices:', top5.tolist())
print('Top-5 values:', values[top5].tolist())

assert result.shape == (1, 1000)
assert result.dtype == np.float32
assert np.isfinite(result).all()
print('RVV256 ResNet-18 validation: PASSED')
PY
```

OBSERVED RESULT Shape (1, 1000); float32; all values finite; predicted index 623; top-five indices [623, 111, 499, 813, 596]; validation PASSED.

6. Results Achieved

Validation item	Verified result	Interpretation
Writable workspace	/shared/school26/ resnet18_iree	User can reproduce the workflow without altering the master project
Compiled module	45 MB VMFB	MLIR compilation completed and produced a deployable artifact
Target settings	RV64 + RVV target; VLEN 256	The module and QEMU run were configured for the intended vector-capable target
Runtime execution	Status 0; EXEC @main_graph	The compatible RISC-V runtime loaded and executed the module
Output file	4.1 KB .npy	The execution path wrote a persistent inference result

Validation item	Verified result	Interpretation
Tensor integrity	(1, 1000), float32, finite	Output matches the expected ResNet-18 classifier interface
Automated gate	PASSED	Shape, type, and finite-value assertions all succeeded

7. Why This Work Is Important

- End-to-end feasibility: it confirms that the model, compiler, VMFB, guest runtime, sysroot, and emulator can operate as one pipeline.
- RISC-V readiness: it establishes a working software path for RVV-capable RISC-V evaluation before deployment on physical hardware.
- Reduced integration risk: the compiler/runtime mismatch was isolated and a compatible runtime was identified, preventing repeated false starts.
- Reproducibility: absolute paths, captured logs, artifact checks, and assertions give students and researchers a repeatable procedure.
- Secure shared-cluster practice: the workflow uses a user-owned directory and avoids broad permission changes to shared projects.
- Correct interpretation: the zero-input test is explicitly separated from model-accuracy claims, preserving scientific validity.

8. Limitations

- The input was an all-zero tensor; class index 623 and the reported top-five indices are not meaningful classification results.
- The successful target configuration does not by itself prove how many RVV instructions were emitted or used inside every kernel; disassembly or compiler diagnostics are required for that claim.
- No scalar-versus-vector latency, throughput, memory, or energy comparison was collected in this session.
- QEMU validates functional behavior but does not represent the timing of a physical RISC-V processor or accelerator.

9. Recommended Next Steps

1. Prepare one real test image using the exact resizing and ImageNet normalization expected by the exported ResNet-18 model.
2. Run the real image through the same VMFB and map the top-five indices to ImageNet labels.
3. Compile and execute a scalar RISC-V baseline with the same input and compare it with the RVV256 configuration.
4. Use repeated warm-up and measured runs to report median latency and variability instead of a single timing.
5. Inspect generated code or disassembly to verify RVV instruction emission and identify vectorized kernels.
6. Record compiler/runtime versions and artifact checksums in the logs for long-term reproducibility.
7. Repeat the validated workflow on physical RISC-V hardware when the target platform becomes available.

10. Conclusion

The ResNet-18 IREE pipeline on the CAID ChipCluster is functionally validated for a RISC-V vector-capable QEMU environment. The critical success factors were use of a writable project directory, verification of binary architecture, selection of the compatible /opt IREE runtime, use of explicit QEMU/sysroot paths, and assertion-based output checks. The resulting procedure now provides a reliable base for real-image classification, scalar-versus-vector benchmarking, and future RISC-V hardware deployment.

WORKING COMBINATION TO RETAIN Compiler: /usr/local/bin/iree-compile | Runtime: /opt/iree-run-module | QEMU: /opt/riscv/bin/qemu-riscv64 | Sysroot: /opt/iree-riscv-toolchain/sysroot